CMU 15-441/641: Networking and the Internet
Project 2 Lab: TCP Meets the Real World

TAs: Ayushi Bansal, Yujing Wang, Yiran Lei, Isabel Suizo, Mingjun Li
Last update: October 30, 2026

Released: October 30th, 2026
Due: 8:00 pm, November 6th, 2026
Late due: 8:00 pm, November 8th, 2026
Lab repo: https://github.com/computer-networks/cmu-tcp-lab

1 Introduction

Congratulations: you built a TCP! Your CMU-TCP shakes hands, slides its window, recovers from loss with fast retransmit, respects the receiver with flow control, and backs off politely with Reno. On the gentle networks of Checkpoints 1 and 2, that is everything a transport protocol needs.

The real Internet is not gentle. In this lab, your TCP is dropped into three networks that have broken real TCPs in practice, the kind of problems that keep engineers at Google, Meta, Microsoft, NVIDIA, and every major cloud provider busy:

- **Bufferbloat at home** (required). Your roommate starts a big Google Drive upload, and your online game becomes unplayable. Nothing is broken and nothing is lost, yet every packet now waits in line behind hundreds of milliseconds of upload.
- **Incast in the datacenter** (required). A search query or a Spark shuffle asks dozens of servers for data *at the same instant*. Their answers collide at one switch port, the shallow buffer overflows, and one unlucky flow waits seconds for a timeout while the whole query waits for it.
- **Packet spraying in an AI cluster** (optional, extra credit). To keep thousands of GPUs busy, modern switches spray the packets of a single flow across many paths. Packets now arrive out of order, and your TCP keeps panicking about losses that never happened.

You are the transport engineer. For each network, you will find out why your Reno struggles, design a fix, implement it on top of your existing CMU-TCP, and get a score based on how well your implementation performs. You start from your Checkpoint 2 implementation: once your design is right, the code is a modest addition on top of it.

Local tests and Gradescope tests. Each challenge comes with a *local test* that you run on your own machine for debugging and testing. Every test gives a score from 0 to 100, the measurements behind it, and tips. On Gradescope, each challenge has *three* harder tests on different networks. A leaderboard shows how your designs stack up against everyone else's, and **the winners get a prize**.

What you submit. The lab has two parts:

- **Code**, graded by the Gradescope tests. **A test score of 60 or more earns all of that test's points**; below 60 you earn points in proportion. A good design reaches 60 without being perfect.
- **A report** (Section 6): for each challenge, your local test score before and after, why Reno struggles, and the design you made.

Groups. You may complete this lab individually or in a group of at most two students, and you can use your own (or either partner's) Checkpoint 2 implementation as the starting point.

2 What You Can Change

The following knobs are enough to solve all three challenges; part of the fun is figuring out which ones matter where.

- **Header extension (`extension_data`).** Carry extra information on data packets or ACKs, such as timestamps, selective acknowledgment blocks, or anything else your design needs.
- **The advertised window.** The receiver can tell each sender exactly how much it may send.
- **When and how much to send.** Your congestion-window rule: how it grows, what makes it shrink, and by how much.
- **When to declare a packet lost.** The duplicate-ACK threshold, time-based rules, and what to retransmit (one packet, a few, or everything outstanding).
- **How the receiver acknowledges.** When to send ACKs, and what to put in them. For example, you can implement selective acknowledgments (SACK): each ACK also lists the out-of-order data the receiver already holds, so the sender knows exactly which packets are missing.
- **Host-level information.** In Incast, `lab_host.h` tells your receiver about all connections on the same host (Section 4).

A few things stay fixed:

- Do not modify `cmu_packet.h`. Your retransmission timer must use `DEFAULT_TIMEOUT`, which is 3 seconds in every test.
- The receiving application must get exactly the bytes that were sent; otherwise the test scores 0.

3 Challenge 1: Bufferbloat (required)

3.1 The scenario

Game night, ruined

It is Friday night. You are one round away from a win in your favorite online game, and your ping is a crisp 20 ms. Then your roommate starts uploading 30 GB of vacation videos to Google Drive. Your ping jumps to 300 ms, your character teleports around the map, and you lose. Nothing was "broken": no cable was cut, and barely any packets were lost. The upload simply filled the router's buffer, and every one of your game packets had to wait in line behind it.
This is **bufferbloat**, and it affects every home network, every coffee-shop Wi-Fi, and every cellular link. Your job: make the upload fast *and* keep the game playable, **using your customized transport for both**.

The setup. Two of our applications run on the same home network and both use *your* CMU-TCP (built from your `bufferbloat/` folder):

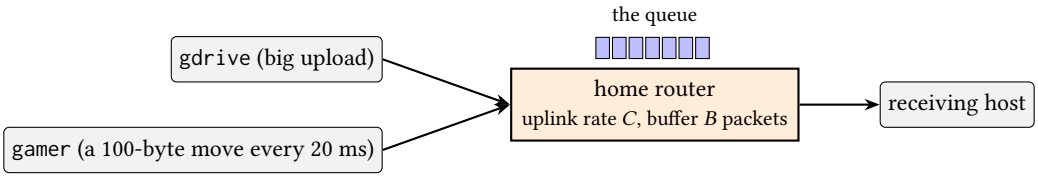


Figure 1: Bufferbloat: an upload and a game share one home uplink with a large buffer.

- `gdrive` uploads a file as fast as your transport allows.
- `gamer` sends a small 100-byte "move" every 20 ms. Each move carries the time it was sent, and the receiving application records when it arrives. That difference is the *lag* the player feels.

Both connections cross the same router uplink (Figure 1). The router has a large buffer, but not an infinite one: if it overflows, packets are dropped.

How we measure success. Your design should get both a fast upload *and* a low lag.

$$\text{Score} \;=\; 100 \times \min\!\left(1,\, \frac{G}{0.95\, C_{\text{avail}}}\right) \times \frac{1}{3} \sum_{p \,\in\, \{20,\,50,\,95\}} \min\!\left(1,\, \frac{L_{\text{target}}}{L_p}\right)$$

- G is `gdrive`'s goodput: file size divided by transfer time. C_{avail} is the available capacity: the uplink rate minus header overhead and `gamer`'s small share. Using 95% of it counts as using all of it.
- L_{20}, L_{50}, L_{95} are the 20th percentile, median, and 95th percentile of the gamer's lag.
- L_{target} is the lag of a well-behaved upload: the one-way propagation delay plus the time to transmit three full-size packets (the one on the wire and about two waiting in the queue, which a busy link needs).

An upload that uses the link and keeps only a couple of packets queued scores 100. Filling the buffer (lag many times the target) or sending timidly (low goodput) lowers your score.

3.2 Test it

Set up first. Before your first test, set up your code by following the lab repository's README (https://github.com/computer-networks/cmu-tcp-lab). Your design goes in `/lab/bufferbloat/`, which `init` created as a copy of your Checkpoint 2 code.

Local test. 4 Mbps uplink, 20 ms round-trip time, 30-packet router buffer; `gdrive` uploads 2 MB while `gamer` plays.

```
./labtest.py run bufferbloat                    # your bufferbloat/ folder
./labtest.py run bufferbloat --build baseline   # your Checkpoint 2 code, for comparison
```

Gradescope tests. Same applications, three different networks:

- **B1, "Slow DSL":** a slower uplink with a longer round-trip time.
- **B2, "Wi-Fi rate drop":** the uplink suddenly becomes several times slower in the middle of the upload.
- **B3, "Route change":** the propagation delay of the path suddenly increases in the middle of the upload.

Each of B1–B3 is worth 10 points, and **a score of 60 or more earns all 10**.

4 Challenge 2: Incast (required)

4.1 The scenario

The query that took three seconds

You type a search query. Behind the scenes, an *aggregator* server asks 40 *worker* servers for their part of the answer, all at the same instant. The workers reply immediately, and 40 replies hit the one switch port that leads to the aggregator at the same moment. Datacenter switches have small buffers, so the buffer overflows in microseconds. Most replies get through, but one worker loses the last packets of its reply. With nothing left to trigger duplicate ACKs, that worker sits idle until its retransmission timer fires. The entire query waits for that one worker.
This is **incast**: many-to-one traffic in datacenters, in search, and the all-to-all phases of AI training. Your job: make every reply arrive as fast as the aggregator's link allows, **without a single timeout**.

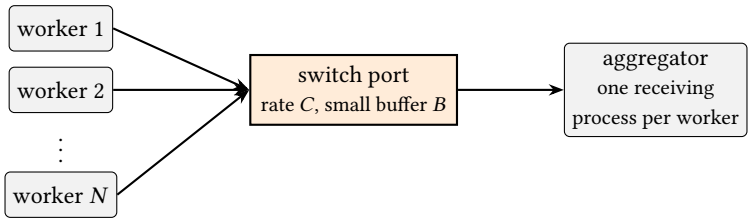


Figure 2: Incast: N workers answer one aggregator at the same moment through one switch port.

The setup. Our application `incast` plays both aggregator and worker, and both use *your* CMU-TCP (built from your `incast/` folder). Each test runs in three phases:

1. **Setup (not timed).** The aggregator runs one receiving process per worker, each with a single listening socket. It records how many bytes it expects on each connection. The workers start a few milliseconds apart and connect with *your* normal three-way handshake (SYN, SYN-ACK, ACK) through the switch. Setup ends when the switch has seen all N handshakes complete. Because workers start staggered and nothing is timed yet, handshakes do not collide.
2. **"Go!" (timed).** All workers are signaled at the same instant. Each calls `cmu_write()` with its whole reply, then `cmu_close()`. The aggregator reads each connection until it has all the expected bytes. The **completion time** T runs from "Go!" until the last byte of the last reply has been read.
3. **Teardown (not timed).** Connections close; how you handle FIN does not affect your score.

Extra information for your receiver: `lab_host.h`

A real aggregator knows things that a single connection cannot see. Your `incast/` folder has a new file, `inc/lab_host.h` (with `src/lab_host.c`), that gives your receiver the same view. **Include the header in the file that needs it** (`#include "lab_host.h"`), then call its functions for information that may help your design, such as how many connections on this host are still receiving and this connection's position among them. Each receiving process is separate; `lab_host.h` is the only way a connection learns about the others. The functions are meant for the **receiving side** (your `TCP_LISTENER` sockets on the aggregator); on a worker's sending socket they return -1 or 0. The comments in `lab_host.h` describe each function. Using it is optional.

How we measure success. Finish fast, and do not waste packets:

$$\text{Score} \;=\; 100 \times \min\!\left(1,\, \frac{1.2\,T^*}{T}\right) \times \min\!\left(1,\, \frac{P_{\text{needed}}}{P_{\text{sent}}}\right)$$

- T is your completion time. T^* is the theoretical minimum: all reply bytes sent back-to-back at the link rate, plus one round-trip time. Finishing within $1.2\,T^*$ (hosts need a little time to process packets) earns full marks.
- P_{needed} is the number of data packets the replies need: each reply's size divided by `MSS`, rounded up, summed over all workers.

- P_{sent} is the number of data packets your senders actually sent, **including retransmissions and packets the switch dropped**.

A design that finishes within $1.2\,T^*$ without a single drop scores 100. Every timeout ruins T, and every unnecessary packet lowers the second term.

4.2 Test it

Your folder. Your design goes in `/lab/incast/`, which `init` created as a copy of your Checkpoint 2 code.

Local test. 8 workers send 64 KB each; 10 Mbps link, 10 ms round-trip time, 24-packet switch buffer.

```
./labtest.py run incast                    # your incast/ folder
./labtest.py run incast --build baseline   # your Checkpoint 2 code, for comparison
```

Gradescope tests. Same application, three different queries:

- **I1, "Bigger fan-in":** more workers, smaller replies, a somewhat larger buffer.
- **I2, "Fan-in beyond the pipe":** so many workers that even one packet each does not fit in the switch.
- **I3, "Mice and elephants":** many small replies and a few large ones in the same query.

Each of I1–I3 is worth 10 points, and **a score of 60 or more earns all 10**.

5 Challenge 3: Packet Spraying (extra credit)

5.1 The scenario

When the network is *too* good at load balancing

An AI-training cluster connects thousands of GPUs with many parallel paths between every pair of switches. Classic datacenters pin each flow to one path, and two big flows that land on the same path get half the bandwidth each while other paths sit idle. For GPUs that cost dollars per minute, that is unacceptable. So modern fabrics (Ultra Ethernet [9], NVIDIA Spectrum-X, AWS SRD [10]) **spray every packet of a flow across all paths**. Load balance becomes nearly perfect.
There is a catch. The paths have slightly different delays, so packets arrive *out of order*. When a path fails, it takes a slice of every window down with it. A TCP that assumes "out of order means lost" panics constantly. Your job: **keep your TCP calm and fast in a sprayed network.**

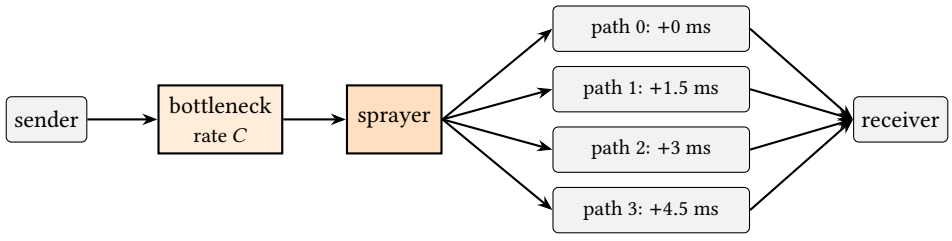


Figure 3: Packet spraying: each packet takes one of several paths, each with its own delay.

The setup. Our application `spray` transfers a file from a sender to a receiver, both using *your* CMU-TCP (built from your `spraying/` folder). Spraying is a **switch feature**: the switch picks a path for every packet on its own, and a retransmitted copy always takes a different path than the copy before it. Your sender does not choose paths, or change how it sends anything. The network is the same in every run: the same packets take the same paths and the same packets are lost.

How we measure success. Fast transfers, no false alarms, and quick repair of real losses:

$$\text{Score} \;=\; 50\min\!\left(1,\frac{G}{0.6\,C_{\text{avail}}}\right) \;+\; 25\min\!\left(1,\frac{\text{Precision}}{0.8}\right) \;+\; 25\min\!\left(1,\frac{2R^{*}}{R_{\text{median}}}\right)$$

- G is your goodput. C_{avail} is the link rate minus header overhead and minus the fraction of packets the network loses. A sender that backs off on real losses cannot use all of it; 60% earns full marks.
- **Precision** is the number of *necessary* retransmissions divided by all retransmissions. A retransmission is unnecessary if an earlier copy of the same bytes reached the receiver. A precision of 0.8 (i.e., at most 20% unnecessary retransmissions) earns full marks.
- R_{median} is the median time to repair a lost packet: from when it was first sent until its bytes first reach the receiver. R^{*} is about one and a half round-trip times plus the delay difference between the fastest and slowest path; repairing within $2R^{*}$ earns full marks.

5.2 Test it

Your folder. Spraying is optional, so `init` does not create its folder. If you attempt it, start from your baseline and put your design in `/lab/spraying/`:

```
cd /lab
cp -r baseline spraying
```

Local test. 10 Mbps bottleneck, 10 ms round-trip time, four paths with +0, +1.5, +3, and +4.5 ms of extra delay; a 2 MB transfer in which 5 consecutive packets (at the 1 MB mark) are lost.

```
./labtest.py run spraying                    # your spraying/ folder
./labtest.py run spraying --build baseline   # your Checkpoint 2 code, for comparison
```

Gradescope tests. Same application, three different fabrics:

- **S1, "Mild spraying, random loss":** the local paths plus a small random loss rate.
- **S2, "Heavy spraying":** much larger delay differences between paths, and ACKs are sprayed too.
- **S3, "Link flap":** one path fails and drops everything during a stretch of the transfer.

Each of S1–S3 is worth 5 extra-credit points, and **a score of 60 or more earns all 5**.

6 Report

Submit one PDF report per group, with one section for each challenge you attempted. Each section answers three questions:

1. **Before and after.** What does the local test score with your Checkpoint 2 implementation (`./labtest.py run <challenge> --build baseline`), and with your new implementation?
2. **Why Reno struggles.** Why do you think your Checkpoint 2 implementation does poorly in this scenario? Point to what you saw in the test output.
3. **Your design.** What did you change, and why does it fix the problem? Use your local and Gradescope results to say where your design works well and where it falls short. An honest analysis of a design that only partly works earns full credit for this part.

7 Submission and Grading

How to submit. Commit your work, then run this in the lab container:

```
./labtest.py package
```

It builds your challenge folders the same way Gradescope does and packs them into `handin.tar.gz` in `/lab`. Upload `handin.tar.gz` to the Gradescope assignment **Project 2 lab: code**, and your report PDF to **Project 2 lab: report**. Each challenge is graded from its own folder (`bufferbloat/`, `incast/`, `spraying/`); a challenge whose folder is missing scores 0.

Grading.

Category	Points	Criteria
Bufferbloat code	30	Gradescope tests B1–B3, 10 points each
Incast code	30	Gradescope tests I1–I3, 10 points each
Bufferbloat report	20	Before and after (4), why Reno struggles (6), your design (10)
Incast report	20	Before and after (4), why Reno struggles (6), your design (10)
Total	**100**	
Spraying code (extra)	+15	Gradescope tests S1–S3, 5 points each
Spraying report (extra)	+10	Before and after (2), why Reno struggles (3), your design (5)

Gradescope runs each test three times on exactly the same network and counts the median score. A score of 60 or more earns all of a test's points; below that, you earn points in proportion:

$$\text{points} = \text{test weight} \times \min\left(1, \frac{\text{test score}}{60}\right).$$

Leaderboard. Every submission appears on the Gradescope leaderboard with its nine test scores (0–100 each).

The winners of the leaderboard get a prize!
Pick a cool name for your algorithm and go for the top.

References

[1] Lawrence Brakmo and Larry Peterson. TCP Vegas: End to End Congestion Avoidance on a Global Internet. *IEEE JSAC*, 1995. Textbook Section 6.4: https://book.systemsapproach.org/congestion/avoidance.html

[2] RFC 6817: Low Extra Delay Background Transport (LEDBAT): https://www.rfc-editor.org/rfc/rfc6817

[3] Neal Cardwell et al. BBR: Congestion-Based Congestion Control. *ACM Queue*, 2016: https://queue.acm.org/detail.cfm?id=3022184

[4] Venkat Arun and Hari Balakrishnan. Copa: Practical Delay-Based Congestion Control for the Internet. *NSDI*, 2018.

[5] Haitao Wu et al. ICTCP: Incast Congestion Control for TCP in Data Center Networks. *CoNEXT*, 2010.

[6] Peter Gao et al. pHost: Distributed Near-Optimal Datacenter Transport Over Commodity Network Fabric. *CoNEXT*, 2015.

[7] Mark Handley et al. Re-architecting Datacenter Networks and Stacks for Low Latency and High Performance. *SIGCOMM*, 2017.

[8] Behnam Montazeri et al. Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities. *SIGCOMM*, 2018.

[9] Ultra Ethernet Consortium: https://ultraethernet.org/ 5.1

[10] Leah Shalev et al. A Cloud-Optimized Transport Protocol for Elastic and Scalable HPC. *IEEE Micro*, 2020. 5.1

[11] RFC 2018: TCP Selective Acknowledgment Options: https://www.rfc-editor.org/rfc/rfc2018

[12] RFC 8985: The RACK-TLP Loss Detection Algorithm for TCP: https://www.rfc-editor.org/rfc/rfc8985

[13] Yuchung Cheng et al. Making Linux TCP Fast. *Netdev 1.2*, 2016.

[14] Textbook Section 6.3 TCP Congestion Control: https://book.systemsapproach.org/congestion/tcpcc.html